Agentic AI Learning Portfolio Report

WIDS-2025 Submission

February 1, 2026

Contents

1 Overview

This report summarizes the learning outcomes and implementations in the WIDS-2025 workspace. The directory contains two assignment sets, weekly agentic AI exercises, and a final project that applies agentic concepts to a YouTube study assistant.

2 Workspace Structure Summary

- **assignment-1/** and **assignment-2/**: Foundational and intermediate experiments with language models and simple agent workflows.
- **W5/**: Google ADK-based agent implementations and tutorials (DataCamp Google ADK).
- **final/youtube_final_project/**: End-to-end application using LLMs and tool pipelines for YouTube learning support.

3 Assignments

3.1 Assignment 1: Core LLM Capabilities

Objective: Explore core NLP tasks using `transformers` pipelines.

- **Summarization (q1.py):** Uses `facebook/bart-large-cnn` to summarize a paragraph, highlighting model-driven condensation and length control.
- **Text Generation (q2.py):** Uses `gpt2` for constrained generation, exploring creativity vs. length with `max_new_tokens`.
- **Sentiment Analysis (q3.py):** Applies a default sentiment pipeline to multiple reviews, returning labels and confidence scores.

Key Learning: Understanding task-specific pipelines and how prompt length, max token count, and model choice impact outputs.

3.2 Assignment 2: Agentic Orchestration with LangGraph

Objective: Build multi-step LLM workflows and route requests using graph-based orchestration.

- **Single-node chat graph (q1.py):** A minimal graph that appends user queries and invokes a local HuggingFace LLM through `LangGraph`.
- **Two-stage QA (q2.py):** A question analyzer rewrites the query, followed by a dedicated answer generator to produce a final response.
- **Router agent (q3.py):** Routes user questions to a `PYTHON` or `GENERAL` agent, then generates the final response based on the chosen path.

Key Learning: Modular agent design, separation of concerns (analysis vs. generation), and conditional branching based on intent.

4 LangChain Framework

4.1 Overview

LangChain is a comprehensive framework for building applications powered by language models. It provides abstractions and components that simplify the development of LLM-powered applications through composable building blocks.

4.2 Core Concepts Used in This Project

- **LLM Wrappers**: Unified interfaces for different model providers (HuggingFace, OpenAI, Ollama)
  - `HuggingFacePipeline`: Wraps HuggingFace transformers pipelines for consistent LangChain integration
  - `langchain-openai`: OpenAI model integration
  - `langchain-ollama`: Local model deployment
- **Chains**: Sequences of LLM calls with preprocessing and postprocessing logic
- **Memory**: Conversation history and state management across agent interactions
- **Vector Stores**: Integration with Chroma for retrieval-augmented generation (RAG)
- **Community Extensions**: `langchain-community` provides additional integrations and utilities

4.3 Application in Assignments

In Assignment 2, LangChain provides:

- Unified model interfaces across different backends
- Prompt templating via `apply_chat_template`
- Stateful conversation management
- Foundation for building agentic workflows

4.4 Key Learning Outcomes

- Understanding LangChain's abstraction layers
- Integrating multiple LLM providers through a common interface
- Building modular, reusable components for LLM applications
- Leveraging vector stores for context-aware responses

5 LangGraph Framework

5.1 Overview

LangGraph is a library for building stateful, multi-actor applications with LLMs. It extends LangChain with graph-based orchestration, enabling complex agent workflows with conditional logic, loops, and parallel execution.

5.2 Core Concepts

- **StateGraph**: Defines the structure of the agent workflow as a directed graph
- **Nodes**: Individual processing units (agents, functions) that transform state
- **Edges**: Connections between nodes defining execution flow
  - *Direct edges*: Unconditional transitions
  - *Conditional edges*: Dynamic routing based on state
- **State**: TypedDict structures that maintain conversation context across the graph
- **Entry/Finish Points**: Define where execution begins and ends

5.3 Implementation Patterns in Assignment 2

5.3.1 Single-Node Chat (q1.py)

- **Pattern**: Linear execution with state accumulation
- **State**: `ChatState` with message list
- **Flow**: Entry → LLM Node → Finish
- **Purpose**: Demonstrates basic graph structure and state management

5.3.2 Two-Stage QA Pipeline (q2.py)

- **Pattern**: Sequential processing with intermediate state
- **State**: `QAState` with `messages`, `clarified_question`, `final_answer`
- **Flow**: Entry → Question Analyzer → Answer Generator → Finish
- **Purpose**: Shows separation of concerns (clarification vs. generation)

5.3.3 Router Agent (q3.py)

- **Pattern**: Conditional routing based on intent classification
- **State**: `RouterState` with `route`, `messages`, `final_answer`
- **Flow**: Entry → Router → [Python Agent — General Agent] → Finish
- **Purpose**: Demonstrates dynamic graph execution and specialized agent delegation

5.4 Advantages of Graph-Based Orchestration

1. **Modularity**: Each node is independently testable and reusable
2. **Flexibility**: Easy to add/remove nodes or modify routing logic
3. **Debuggability**: Clear visualization of execution paths
4. **State Management**: Explicit state transitions and transformations
5. **Scalability**: Support for complex workflows with cycles and parallel branches

5.5 Key Learning Outcomes

- Designing agent workflows as directed graphs
- Managing state across multiple processing steps
- Implementing conditional routing and intent-based delegation
- Building maintainable, modular agent architectures
- Understanding the relationship between LangChain's abstractions and LangGraph's orchestration

6 Weekly Work: Google ADK (W5)

Theme: Building and understanding agent types using Google ADK and Gemini models.

6.1 DataCamp Google ADK Tutorial

The DataCamp module provides a structured walkthrough of ADK capabilities, emphasizing a code-first approach for agent definition and orchestration.

- **Welcome Agent:** Demonstrates basic agent setup and session memory.
- **Sequential Agent:** Shows chain-of-agents execution for decomposing tasks.
- **Parallel Agent:** Highlights concurrent agent execution with independent outputs.
- **Session Agent:** Introduces session memory patterns and the concept of session state services.
- **Structured Outputs:** Shows how output schemas enforce structured responses.
- **Tool Agent:** Demonstrates tool calling and external action execution.

6.2 Custom Agent Exercises

In **W5/tool_agent** and **W5/Welcome_Agent**, custom agents are created using Google ADK. These scripts focus on:

- Defining tool functions (e.g., a contact lookup tool).
- Configuring model settings (temperature, token limits).
- Creating simple agent instructions and behavior policies.

Key Learning: The ADK model of agents extends `BaseAgent`, with explicit tools, configurable behavior, and deployment-ready patterns.

7 Final Project: YouTube Study Assistant

7.1 Project Goal

Build a study assistant that converts YouTube videos or playlists into structured learning materials: notes, flashcards, quizzes, and a question-answering interface.

7.2 Architecture

- **UI:** Streamlit front-end for URL input, content generation, and RAG-style Q&A.
- **Transcript Extraction:** Uses `youtube-transcript-api` for transcripts and `yt-dlp` for playlist parsing and title retrieval.
- **LLM Core:** Gemini model (`gemini-2.5-flash`) via Google Generative AI SDK for summarization, flashcards, and quiz generation.
- **RAG-style QA:** Assembles context from multiple transcripts and answers questions strictly grounded in the extracted transcripts.

7.3 Key Components

Module	Purpose
`streamlit_app.py`	Front-end UI: URL input, summary/flashcard/quiz generation, and chat-like Q&A.
`core/transcript.py`	Fetches transcript text from a video ID.
`core/playlist.py`	Extracts video IDs from a playlist.
`core/youtube_utils.py`	URL parsing, playlist detection, and title retrieval.
`core/summarizer.py`	Generates chapter-wise notes from transcripts.
`core/flashcards.py`	Generates concise Q/A flashcards from transcripts.
`core/quiz.py`	Generates multiple-choice questions.
`core/rag.py`	Answers user questions using transcript-based context.
`core/llm.py`	Loads API keys and dispatches LLM calls.

7.4 Process Flow (Conceptual)

1. User provides a YouTube video or playlist URL.
2. System extracts video IDs and fetches transcripts.
3. User triggers:
   - Chapter-wise notes,
   - Flashcards,
   - Multiple-choice quiz.
4. User asks questions across all processed videos; responses are grounded in transcript context.

7.5 Key Learning Outcomes

- Building an end-to-end agentic workflow with UI, tools, and LLM output.
- Prompt engineering for structured study materials.
- Multi-source context assembly for RAG-style Q&A.
- Managing session state within a front-end application.

8 Tooling and Dependencies

Primary Libraries:

- **Transformers, PyTorch, Accelerate**: local model execution and pipeline inference.
- **LangChain Ecosystem**:
  - `langchain`: Core framework for LLM application development
  - `langchain-core`: Foundation abstractions and interfaces
  - `langchain-huggingface`: HuggingFace model integration
  - `langchain-openai`: OpenAI model support
  - `langchain-ollama`: Local model deployment
  - `langchain-chroma`: Vector store integration for RAG
  - `langchain-community`: Community-contributed extensions
- **LangGraph**: Graph-based agent orchestration with state management, conditional routing, and complex workflow support.
- **Google Generative AI SDK**: Gemini model access for generation tasks.
- **Streamlit**: interactive UI for the final application.
- **YouTube Transcript API, yt-dlp**: transcript extraction and playlist processing.

9 Summary of Learning

The repository demonstrates a progression from foundational LLM tasks to multi-step agent orchestration, and finally to a full application that integrates tools, retrieval context, and structured outputs. Core competencies developed include:

- Prompt design for summarization, generation, and classification.
- Agentic workflow design with routing, sequencing, and state management using LangGraph.
- LangChain framework proficiency: model abstraction, prompt templating, and component composition.
- Graph-based orchestration: building stateful multi-agent systems with conditional logic.
- Tool integration for real-world data retrieval (YouTube transcripts, playlists).

- Comparing frameworks: understanding trade-offs between LangChain/LangGraph and Google ADK.
- Building a user-facing application that packages agentic AI capabilities into a usable learning tool.

9.1 Framework Comparison: LangChain/LangGraph vs. Google ADK

Aspect	LangChain/LangGraph	Google ADK
Philosophy	Composable chains and graph-based workflows	Code-first agent definitions with Google Cloud integration
State	TypedDict with explicit state management	Session services (in-memory, database, Vertex AI)
Routing	Conditional edges in graphs	Conditional agent selection and tool calling
Tools	Extensive ecosystem of pre-built tools	Google Cloud native tools + custom function tools
Deployment	Framework-agnostic (requires separate deployment setup)	Built-in Google Cloud deployment support
Learning Curve	Steeper (many abstractions)	Gentler (straightforward Python classes)
Use Case	Flexible, multi-provider applications	Google Cloud-centric production systems